

Professors Francesco Massel, Leon Loveridge, and Kjetil Børkje
University of South-Eastern Norway
Introduction to quantum computing (Graduate)

Xanadu learning content suggestions

The following content suggestions include some theory and practice on gate-based quantum computing in line with the current concepts taught in the course. The students will make the most of the material if they go through both the theory and the coding exercises.

The content marked as “Explanation” corresponds to text and image explanations of the concepts. The content marked as “Codebook” includes interactive coding exercises. The content marked as “Demo” includes both explanation and code. In demos, the code cannot be executed directly on the page but it can be downloaded as a notebook or Python file. The content marked as “Video” links to a video on our YouTube channels that provides a walkthrough of a given topic. Any of this content can be used to prepare the lectures or as practice or reference material for the students. Each of the Codebook submodules should take around 20-30min to complete plus some additional time for the coding exercises.

For instructions on installing PennyLane, please see instructions [here](#) (at the end of this document).

Lecture 1 – Introduction to quantum computing and quantum circuits

- [What is quantum computing?](#) (Explanation)
- Qubits, bra-ket notation, superposition, and measurements ([Codebook Module I.1](#)).
- Quantum Circuits ([Codebook Module I.2](#)).
- Unitaries ([Codebook Module I.3](#)).

Lecture 2 – Single-qubit gates, rotation gates, and the Bloch sphere

- Quantum operations and 1-qubit gates ([Codebook Module I.4](#)).
- Global and relative phases ([Codebook Module I.5](#)).
- Rotation gates and Bloch sphere ([Codebook Module I.6](#)).
- Universal gate sets ([Codebook Module I.7](#)).

Lecture 3 – Quantum state preparation, measurements, and expectation values

- Quantum state preparation ([Codebook Module I.8](#)).
- Projective measurements, basis, and basis rotations ([Codebook Module I.9](#)).
- Observables and expectation values ([Codebook Module I.10](#)).

Lecture 4 – Introduction to multi-qubit circuits and entanglement

- Tensor product and multi-qubit systems ([Codebook Module I.11](#)).
- Entanglement and controlled operations ([Codebook Module I.12](#)).
- The controlled-Z, Toffoli, and SWAP gates ([Codebook Module I.13](#)).
- Bell state and bell basis ([Codebook Module I.14](#)).

Lecture 5 – Introduction to quantum algorithms: teleportation, introduction to oracles

- [Quantum teleportation](#) (Demo) – Basic example of a sequence of operations (i.e., an algorithm) that results in a useful outcome.
 - [Teleporting qubits with entanglement](#) (Video)
- Exponential speedups ([Codebook Module A.1](#)) – Explains why quantum superposition is not enough to explain the increased computational power of quantum computers.
- Introduction to oracles ([Codebook Module A.2](#)) – Illustrates a simple oracle that does no better than the classical brute force approach.
- Pair programming ([Codebook Module A.3](#)) – Improves on simple oracle from Module A.2 by using relative phases in a superposition of states.

Lecture 6 – Introduction to quantum algorithms: Deutsch-Jozsa and Bernstein-Vazirani

- Global information and promises and how they are useful for algorithm design ([Codebook Module A.4](#)).
- The Hadamard transform ([Codebook Module A.5](#)).
- The Deutsch-Jozsa algorithm ([Codebook module A.6](#)).
- Additional resources:
 - [Qutrits and quantum algorithms](#) (Demo) – The Bernstein-Vazirani algorithm using qutrits and qutrits.
 - [Qutrits and the Deutsch-Jozsa Algorithm](#) (Community Demo)

Lecture G1 – Introduction to Grover’s algorithm – part 1

- Introduction to Grover’s algorithm: Amplitude amplification and the diffusion operator ([Codebook Module G.1](#)).
- The geometry of Grover search ([Codebook Module G.2](#)).
- Phase kickback and implementing diffusion and oracle operators using multi-controlled X gates ([Codebook Module G.3](#)).

Lecture G2 – Introduction to Grover’s algorithm – part 2

- Grover step and scaling ([Codebook Module G.4](#)).
- Geometry and scaling of Grover search for multiple solutions ([Codebook Module G.5](#)).
- Additional resources:
 - [Grover’s algorithm](#) (Demo) – Gives a brief overview of Grover’s algorithm.
 - [Grover’s algorithm in PennyLane](#) (Video).
 - [Introduction to amplitude amplification](#) (Demo).

Lecture F – The Quantum Fourier Transform (QFT) algorithm

- Intro to/Motivation for Quantum Fourier Transform (QFT) ([Codebook Module F.1](#)).
- The QFT for one and two qubits and the properties of the QFT ([Codebook Module F.2](#)).
- The QFT as a generalization of the Hadamard transform, the n-qubit QFT circuit, and applications. ([Codebook Module F.3](#)).
- Additional resource:
 - [Introduction to the QFT](#) (Demo).

Lecture P – The Quantum Phase Estimation (QPE) algorithm

- Phase kickback and QPE Applications ([Codebook Module P.1](#)).
- The QPE circuit ([Codebook Module P.2](#)).
- QPE for phases that do not have a t-bit binary expansion ([Codebook module P.3](#)).
- Outcomes of the QPE subroutine when the target wires are prepared in an arbitrary state ([Codebook module P.4](#)).
- Additional resource:
 - [Building a quantum lock using phase kickback](#) (Demo).
 - [Introduction to the QPE algorithm](#) (Demo).

Lecture S1 – Shor’s algorithm: Motivation and the period-finding algorithm

- Modular arithmetic ([Codebook Module S.1](#)).
- Nontrivial square roots in modular arithmetic ([Codebook Module S.2](#)).
- The period-finding algorithm ([Codebook Module S.3](#)).

Lecture S2 – Shor’s algorithm with an application to the RSA encryption system

- The sequential structure of Shor's algorithm ([Codebook Module S.4](#)).
- The RSA encryption system ([Codebook Module S.5](#)).

Lecture N1 - Introduction to noise and mixed states

- Density operator and mixed states ([Codebook Module N.1](#)).
- Operations with mixed states ([Codebook Module N.2](#)).

Lecture N2 - Partial trace and noise channels

- Partial trace and purification ([Codebook Module N.3](#)).
- [Noisy circuits](#) (Demo) – Introduction to noisy channels, using Kraus operators and simulating noisy circuits.

Lecture E – Introduction to quantum error correction

- [Introduction to quantum error correction](#) (Explanation).
- [From NISQ to ISQ](#) (Blog).
- Motivation for error correction and repetition codes ([Codebook Module E.1](#)).
- Bit flip and phase errors ([Codebook Module E.2](#)).
- Shor’s 9-qubit code ([Codebook Module E.3](#)).

Support and additional resources

- [Discussion Forum](#): Where students can ask questions.
 - [Xanadu Slack](#): Where students can join the community.
 - [PennyLane Certificate](#): Certification for students to improve their skills in quantum programming as well as earn the rightful industry recognition associated with it.
 - [PennyLane challenges](#): For additional practice ideas.
 - [Xanadu research](#): Contains research on quantum machine learning, quantum chemistry, photonic quantum computing, quantum software, fault-tolerant architectures, and more.
 - [Xanadu newsletter](#): to stay up to date with the latest content and news.
-

About Xanadu and PennyLane

[Xanadu](#) is a Canadian quantum computing company with the mission to build quantum computers that are useful and available to people everywhere. Xanadu is one of the world's leading quantum hardware and software companies and also leads the development of PennyLane.

[PennyLane](#) is an open-source software framework for quantum machine learning, quantum chemistry, and quantum computing, with the ability to run on all hardware.

PennyLane installation instructions

Option 1: Easiest

The easiest way to use PennyLane is using [Google Colab](#). Open a new notebook there and run the following line in the first cell of your notebook (including the exclamation mark):

```
!pip install pennylane
```

Option 2: For heavy workloads

If you're running large workloads it's best to install PennyLane locally. While you can simply run **pip install pennylane**, it's strongly recommended that you create a new virtual environment to avoid any installation issues. You can create a virtual environment with Conda and install PennyLane as follows:

1. Install Miniconda following the instructions [here](#).
2. Open your terminal (mac) or command line (Windows).
3. Create a new Conda environment with: **conda create --name <name_of_your_environment> python=3.10**
4. Activate the environment with: **conda activate <name_of_your_environment>**
5. Install PennyLane with: **python -m pip install pennylane**
6. Install other useful packages with: **python -m pip install pennylane jupyter matplotlib**

Note that you will be installing 3 packages here: PennyLane, Jupyter, and Matplotlib. Also, note that where it says **<name_of_your_environment>** you can choose any name that you want. After this you can write **jupyter notebook** in your terminal/command line and you will be ready to create programs using PennyLane.

Every time you install a package remember to install it with **python -m pip install** instead of **conda install**.

You can also watch the video [here](#) for detailed PennyLane installation instructions.

If you have trouble installing PennyLane locally, please let us know in our [discussion forum](#).